Тема 4. Алгоритмы сортировки и поиска на массивах

Лекция 7, 8

Методы улучшения алгоритмов сортировок

Целью данной темы является изучение алгоритмов сортировки и поиска элементов массива. В ней осуществляется постановка задачи сортировки, формулируются свойства алгоритмов сортировки, рассматриваются базовые алгоритмы: сортировка обменом, вставками и выбором, а так же их различные улучшения, позволяющие существенно сократить их временную сложность. Так же, в этой теме осуществляется постановка задачи поиска и рассматриваются алгоритмы линейного, блочного и бинарного поиска. Проводится сравнительный анализ всех рассмотренных алгоритмов.

4.4. Методы улучшения алгоритмов сортировок

Улучшение сортировки обменом. Во-первых, рассмотрим ситуацию, когда на каком-либо из проходов не произошло ни одного обмена. Что это значит? Это значит, что все пары расположены в правильном порядке, так что массив уже отсортирован. И продолжать процесс не имеет смысла (особенно, если массив был отсортирован с самого начала). Итак, первое улучшение алгоритма заключается в запоминании, производился ли на данном проходе какой-либо обмен. Если нет – алгоритм заканчивает работу.

Процесс улучшения можно продолжить, если запоминать не только сам факт обмена, но и индекс последнего обмена k. Действительно: все пары соседних элементов с индексами, меньшими k, уже расположены в нужном порядке. Дальнейшие проходы можно заканчивать на индексе k, вместо того чтобы двигаться до установленной заранее верхней границы i. На этом основано второе улучшение алгоритма (см. рис. 4.11). В нем введена логическая переменная *flag* – признак наличия хотя бы одного обмена в проходе.

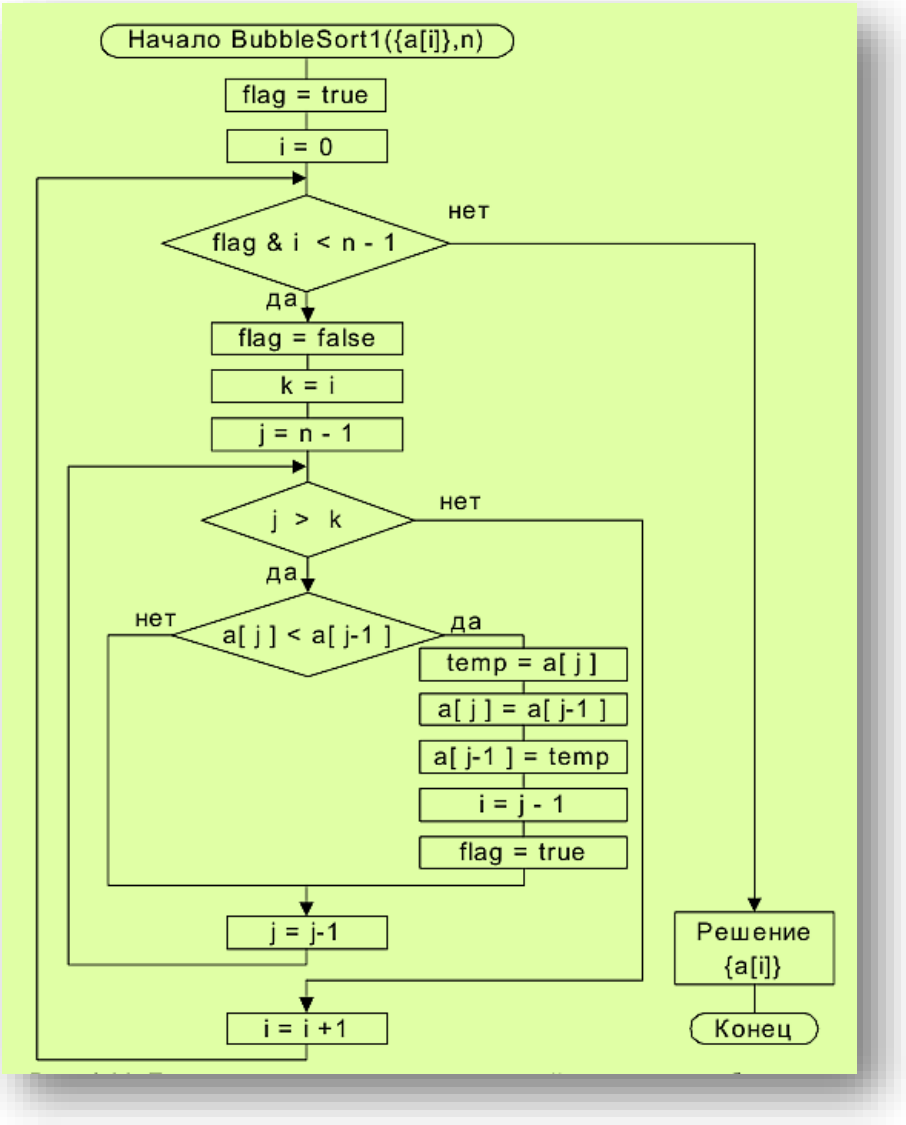


Рис. 4.11. Блок-схема алгоритма улучшенной сортировки обменом

Качественно другое улучшение алгоритма сортировки обменом можно получить из следующего наблюдения. Хотя легкий пузырек снизу поднимется наверх за один проход, тяжелые пузырьки опускаются с минимальной скоростью: один шаг за итерацию. Так что массив 2 3 4 5 6 1 будет отсортирован за 1 проход, а сортировка последовательности 6 1 2 3 4 5 потребует 5 проходов.

Чтобы избежать подобного эффекта, можно менять направление следующих один за другим проходов. Получившийся алгоритм иногда называют "***шейкер-сортировкой***". На рис. 4.12 приведена его блок-схема.

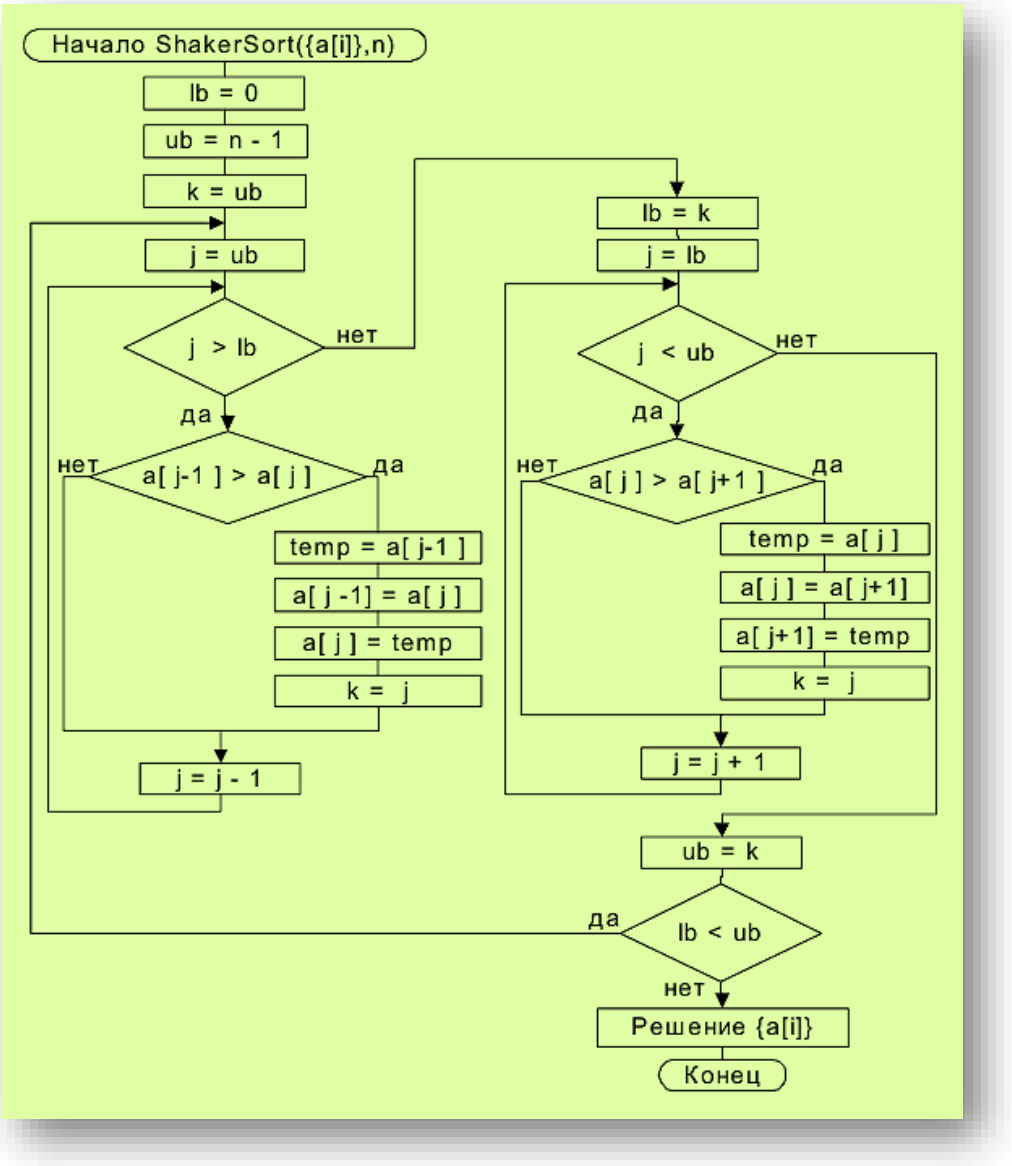


Рис. 4.12. Блок схема алгоритма шейкер-сортировки

Насколько описанные изменения повлияли на эффективность метода? В худшем случае эти виды улучшенной сортировки не отличаются от исходного алгоритма. Среднее количество сравнений, хоть и уменьшилось, но остается $O(n^2)$, в то время как число обменов не поменялось вообще

никак. В лучшем случае, когда массив уже упорядочен, потребуется всего один проход и *n-1* сравнение, что составляет *O(n).* Перестановки в этом случае не выполняются. Дополнительная память, очевидно, не требуется. Поведение усовершенствованных алгоритмов довольно естественное, почти отсортированный массив будет отсортирован намного быстрее случайного. Сортировка пузырьком и первые ее улучшения устойчивы, однако шейкер-сортировка утрачивает это качество.

Интересным ***улучшением метода сортировки обменом*** является так называемая "***быстрая сортировка***", предложенная Ч. Хоаром. Метод основан на подходе "разделяй и властвуй". Общая схема такова:

1) из массива выбирается некоторый опорный элемент *pivot = a[i];*

2) запускается процедура разделения массива, которая перемещает все ключи, меньшие, либо равные *pivot*, влево от него, а все ключи, большие, либо равные *pivot* – вправо, элемент *pivot* размещается в центре и уже находится на правильном месте в отсортированной последовательности;

3) теперь массив состоит из трех подмножеств, как показано на рис. 4.13, причем левое подмножество содержит элементы меньшие, либо равные элементам правого;

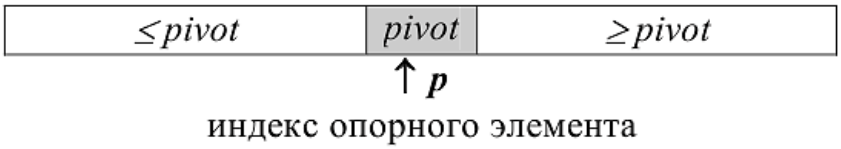


Рис. 4.13. Результат разделения массива

4) для левого и правого подмассивов: если в подмассиве не менее двух элементов, рекурсивно запускаем для него ту же процедуру сортировки.

В итоге получится полностью отсортированная последовательность.

На рис. 4.14 представлена блок-схема алгоритма быстрой сортировки. В отличие от ранее рассмотренных алгоритмов сортировки его параметрами являются сортируемая последовательность *{a[i]}* и индексы ее первого и

последнего элемента – *lb* (lower bound - нижняя граница) и *ub* (upper bound - верхняя граница) соответственно.

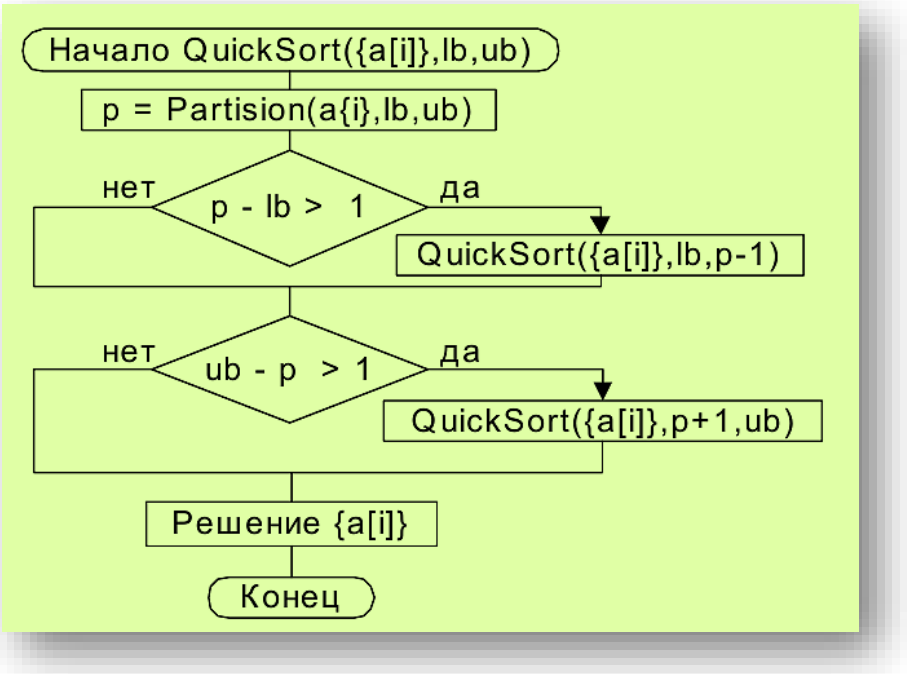


Рис. 4.14. Блок-схема алгоритма быстрой сортировки

Рассмотрим алгоритм разделения массива подробнее. Его входными параметрами являются разделяемый массив *{a[i]}* и индексы ее первого (*lb*) и последнего элемента (*ub*).

1. Выберем опорный элемент *pivot*, например, из середины разделяемой части: *pivot=a[lb+(ub-lb)/2]*.

2. Исключим элемент *pivot* из разделяемой последовательности, вставив вместо него *a[lb]*. Введем два указателя: *i* и *j*. В начале алгоритма они содержат, соответственно, индекс элемента, следующего за левым крайним, и индекс правого крайнего элемента последовательности.

3. Будем двигать указатель *i* с шагом в 1 по направлению к концу массива, пока $i \le j$ и не будет найден элемент $a[i] \ge pivot$. Затем аналогичным образом начнем двигать указатель *j* от конца массива к началу, пока $i \le j$ и не будет найден $a[j] \le pivot$.

4. Далее, если $i \leq j$, меняем *a[i]* и *a[j]* местами и продолжаем двигать *i, j* по тем же правилам.

5. Повторяем шаги 3 – 4, пока $i \leq j$.

6. По завершению разделения указатель *j* содержит индекс последнего элемента части последовательности, содержащей элементы не превосходящие *pivot*. Переместим *a[j]* на первое место (*a[lb]=a[j]).* На место элемента *a[j]* вставим pivot (*a[j]=pivot*).

После завершения шага 6 исходная последовательность окажется разделенной на три части, как показано на рис. 4.13. Элементы, не превосходящие *pivot*, имеют индексы из диапазона *lb…j – 1*, элемент *pivot* имеет индекс *p = j*, а элементы не меньшие *pivot* имеют индексы *j+1…ub*.

Рассмотрим работу процедуры для массива *a[0]...a[6]* и опорного элемента *pivot = a[3]*. Рис. 4.15 иллюстрирует разделение для этого примера.

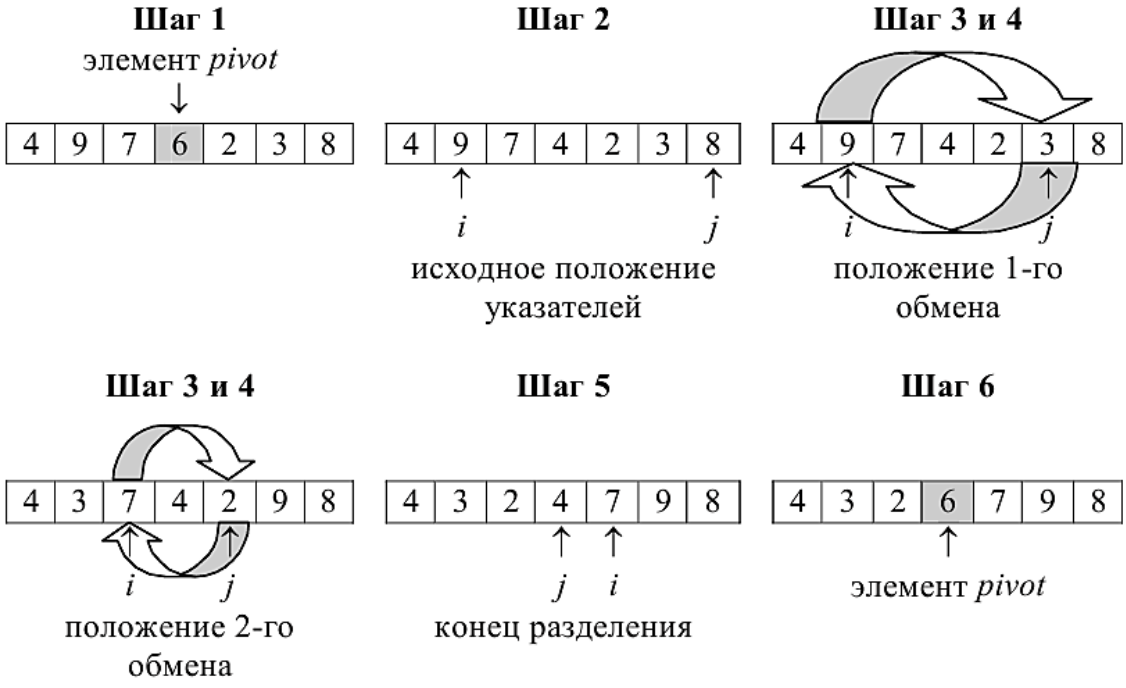


Рис. 4.15. Пример разделения массива

На шаге 6 массив разделен на три части: все элементы левой меньше либо равны *pivot*, все элементы правой – больше, либо равны *pivot*, средняя часть содержит только *pivot*. Разделение завершено.

На рис. 4.16 приведена блок-схема алгоритма разделения массива.

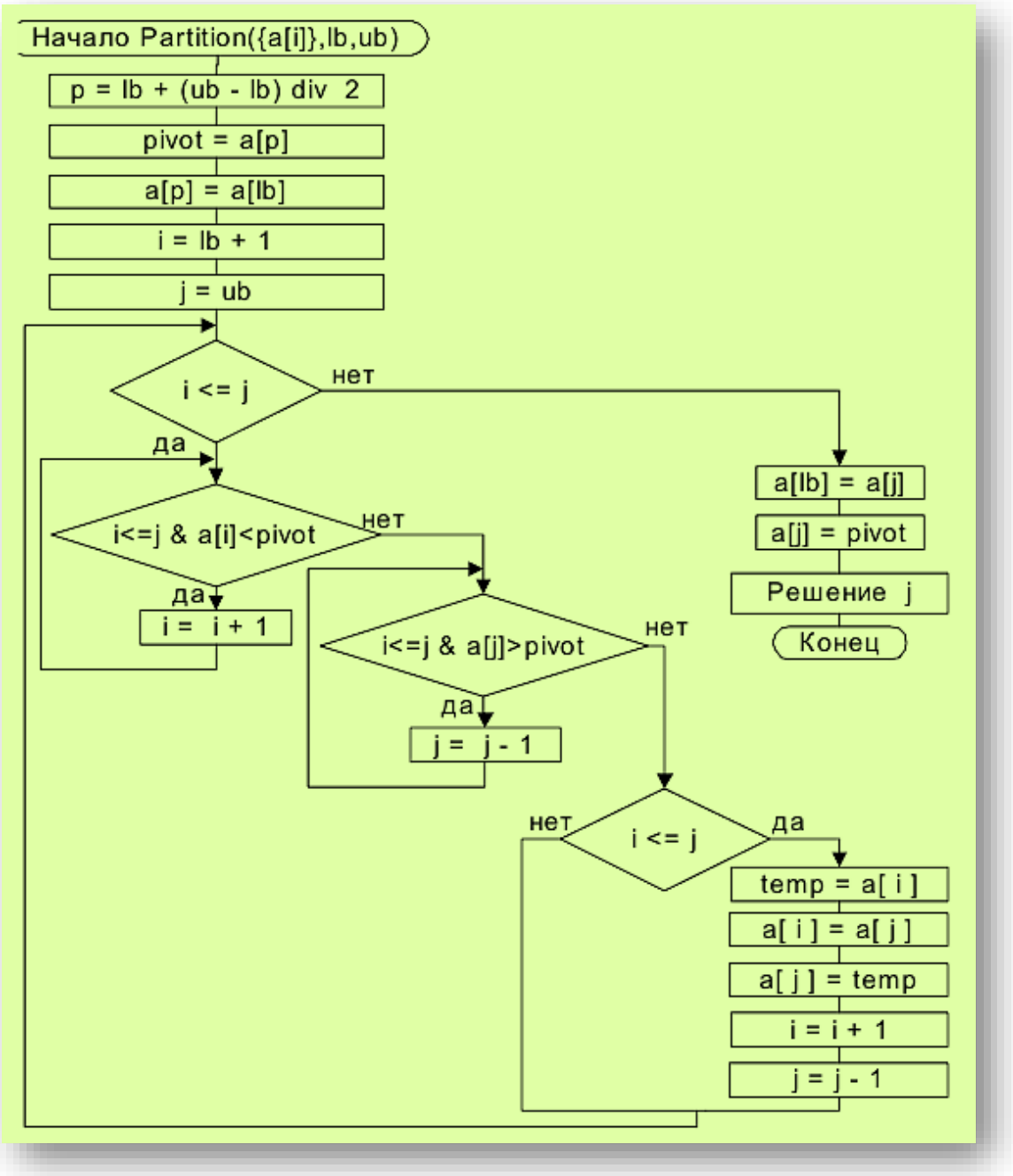


Рис. 4.16. Блок-схема процедуры разделения массива

Очевидно, что временная сложность каждого разделение имеет порядок *O(n).* Количество шагов деления (глубина рекурсии) составляет приблизительно $log_2 n$, если массив делится на более-менее равные части. Таким образом, общая временная сложность алгоритма быстрой сортировки составляет *O(n log n)*, что и имеет место на практике.

Однако, возможен случай таких входных данных, на которых алгоритм будет иметь временную сложность порядка $O(n^2)$. Такое происходит, если каждый раз в качестве опорного элемента выбирается максимум или минимум входной последовательности. Например, такая ситуация имеет место, если массив уже упорядочен, или упорядочен в обратной последовательности, а в качестве опорного элемента выбирается первый или последний элемент последовательности. Поэтому рекомендуется выбирать опорный элемент из середины последовательности или вообще – случайным образом. Если опорный элемент выбирается случайно, вероятность выбора минимального (максимального) элемента этого равна *1/n*. И эта вероятность должна реализовываться на каждом шаге... Вообще говоря, малореальная ситуация.

Метод неустойчив. Поведение довольно естественно, если учесть, что при частичной упорядоченности повышаются шансы разделения массива на более равные части.

Сортировка использует дополнительную память, так как приблизительная глубина рекурсии составляет $log_2 n$, а данные о рекурсивных подвызовах каждый раз добавляются в стек.

Улучшение сортировки вставками. Алгоритм сортировки вставками можно слегка улучшить. Заметим, что на каждом шаге внутреннего цикла проверяются 2 условия. Можно объединить их в одно, поставив в начало массива специальный сторожевой элемент. Он должен быть заведомо меньше всех остальных элементов массива (см. пример на рис. 4.17).

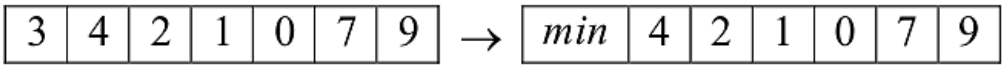


Рис. 4.17. Вставка сторожевого элемента

Тогда при $j = 0$ будет заведомо верно $a[0] \le x$. Цикл остановится на нулевом элементе, что и было целью условия $j \ge 0$. Таким образом, сортировка будет происходить правильным образом, а во внутреннем цикле станет на одно сравнение меньше. С учетом того, что оно производилось n^2 раз, это – реальное преимущество. Однако, отсортированный массив будет не полон, так как из него исчезло первое число. Для окончания сортировки это число следует вернуть назад, а затем вставить в отсортированную последовательность $a[1]...a[n]$ как показано на рис. 4.18.

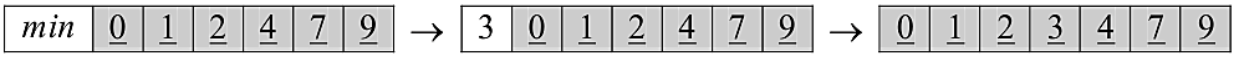


Рис. 4.18. Замена сторожевого элемента на $a[0]$ и досортировка массива

На рис. 4.19 приведена блок-схема *алгоритма сортировки вставками со сделанным улучшением.*

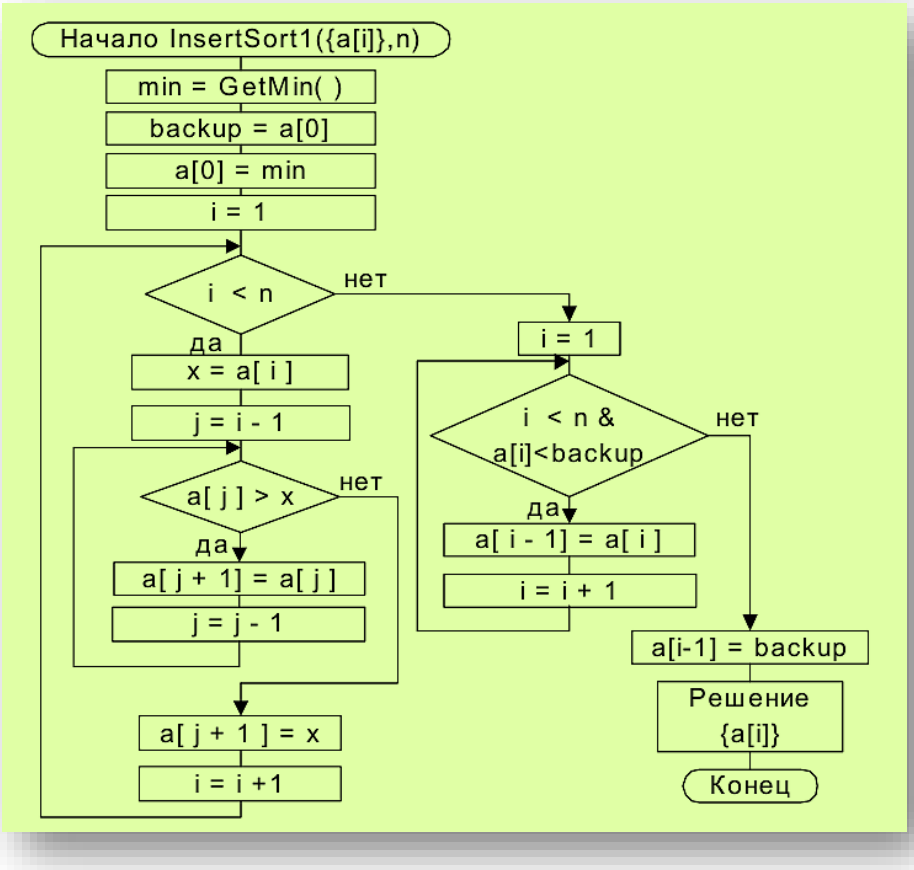


Рис. 4.19. Блок-схема улучшенного алгоритма сортировки вставками

В этом алгоритме используется функция *GetMin( )*, возвращающая элемент, заведомо меньший (меньший или равный, если говорить точнее) всех возможных элементов массива. Эта функция должна быть определена пользователем.

Данное улучшение хотя и делает сортировку более эффективной, но ни сколько не влияет на основные характеристики алгоритма: его временная сложность все так же составляет $O(n^2)$, метод является устойчивым и естественным.

Дальнейшим развитием метода сортировки с включениями является ***сортировка методом Шелла***, называемая по-другому ***сортировкой включениями с уменьшающимся расстоянием***. Не будем описывать алгоритм в общем виде, а ограничимся случаем, когда число элементов в сортируемом массиве является степенью числа 2. Для массива с 2^n элементами алгоритм работает следующим образом. На первой фазе производится сортировка включением всех пар элементов массива, расстояние между которыми есть $2^{(n-1)}$. На второй фазе производится сортировка включением элементов полученного массива, расстояние между которыми есть $2^{(n-2)}$. И так далее, пока не дойдем до фазы с расстоянием между элементами, равным единице, и не выполним завершающую сортировку с включениями.

Рассмотрим пример сортировки массива *a[0].. a[15]* методом Шелла (см. рис. 4.20).

Очевидно, лишь последняя сортировка в рассмотренном примере необходима, чтобы расположить все элементы по своим местам. Так зачем нужны остальные? На самом деле они продвигают элементы максимально близко к соответствующим позициям, так что в последней стадии число перемещений будет весьма невелико. Последовательность и так почти отсортирована. Ускорение подтверждено многочисленными исследованиями и на практике оказывается довольно существенным.

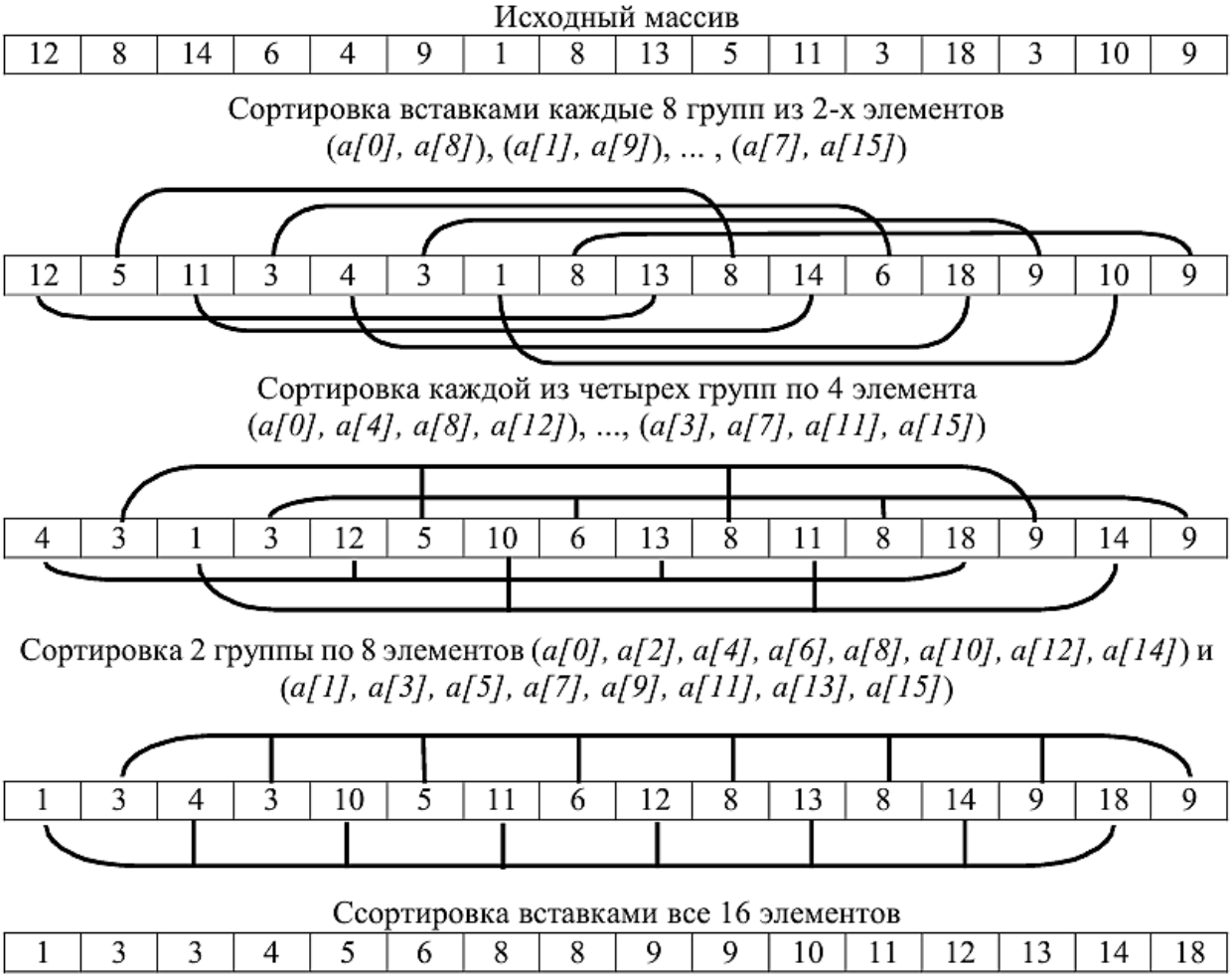


Рис. 4.20. Пример сортировки методом Шелла

Единственной характеристикой сортировки Шелла является приращение – расстояние между сортируемыми элементами, в зависимости от прохода. В конце приращение всегда равно единице – метод завершается обычной сортировкой вставками, но именно последовательность приращений определяет рост эффективности. Использованный в примере набор расстояний 8, 4, 2, 1 – неплохой выбор, особенно, когда количество элементов – степень двойки. В общем случае алгоритм Шелла естественно переформулируется для заданной последовательности из t расстояний между элементами $h_1, h_2, ..., h_t$, для которых выполняются условия $h_t = 1$ и $h_{k-1} > h_k$. Кнут предложил выбирать шаги из ряда обратного следующему: 1, 4, 13, 40, 121,…, а вычислять их по следующей формуле: $h_t = 1$; $h_k = 3 \cdot h_{k+1} + 1$, где $k = 1 \ldots t-1$; $t = \lfloor \log_3 n \rfloor - 1$; $\lfloor \log_3 n \rfloor -$ наибольшее целое, не превосходящее

$log_3 n$; n – размерность исходного массива. На рис. 4.21 приведена блок-схема алгоритма сортировки Шелла, с выбором шагов, предложенных Кнутом.

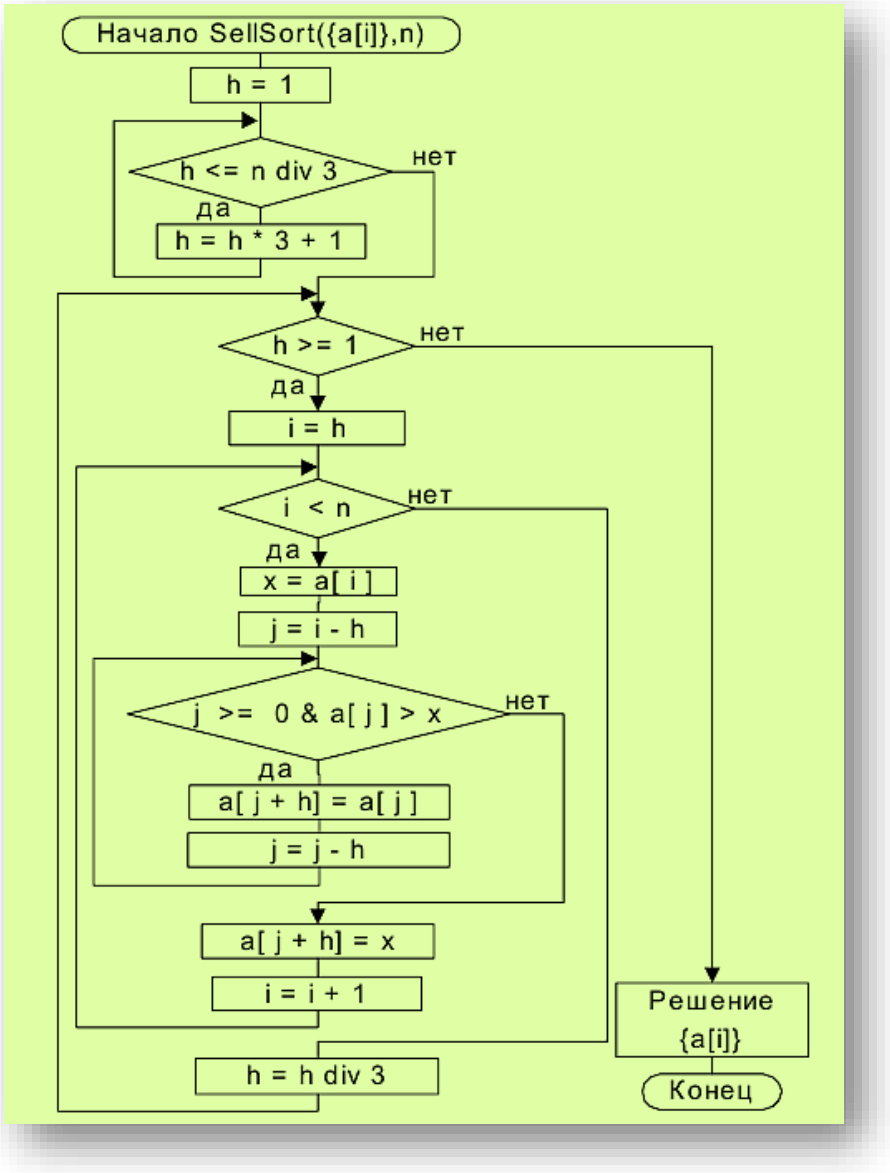
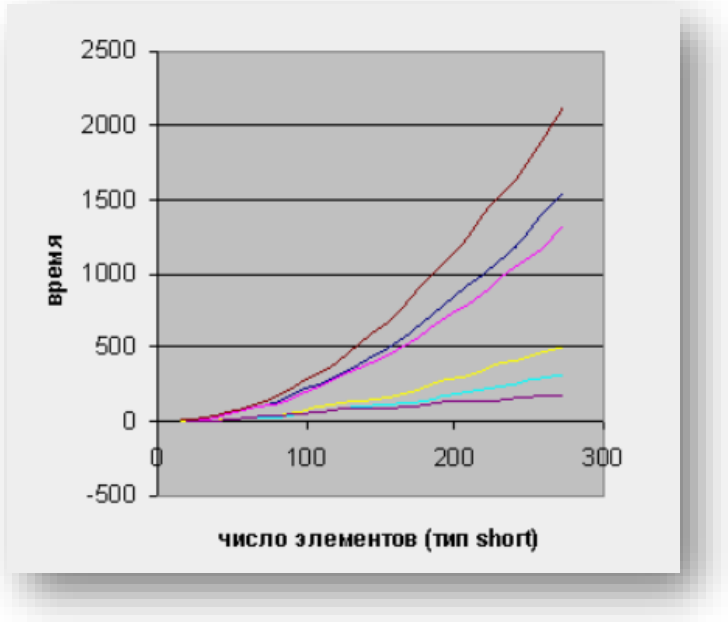


Рис. 4.21. Блок-схема алгоритма сортировки Шелла

Кнут показал, что при последовательности расстояний между элементами …, 121, 40, 13, 4, 1 количество операций сравнения при выполнении алгоритма Шелла даже в наихудшем случае имеет порядок $O(n^{1.5})$. Для случайной последовательности при $n < 60000$ временная сложность алгоритма имеет порядок $O(n^{1.25})$.

Сортировка Шелла естественна, но в отличие от сортировок вставками неустойчива.

Изображенный ниже график иллюстрирует разницу в эффективности изученных алгоритмов.



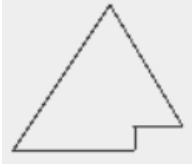
- коричневая линия: сортировка пузырьком;
- синяя линия: шейкер-сортировка;
- розовая линия: сортировка выбором;
- желтая линия: сортировка вставками;
- голубая линия: сортировка вставками со сторожевым элементом;
- фиолетовая линия: сортировка Шелла.

Улучшением сортировки выбором является так называемая ***пирамидальная сортировка***. В основу этого метода положена особая организация данных в массиве, называемая пирамидой.

Пирамида (Heap) – это бинарное дерево высоты k, удовлетворяющая следующим требованиям:

1) узлами дерева являются элементы массива;

2) дерево является сбалансированным, т.е. все листья имеют глубину k или $k - 1$, при этом уровень $k - 1$ полностью заполнен, а уровень k заполнен слева направо, т.е форма пирамиды имеет приблизительно такой вид:



3) выполняется "свойство пирамиды": каждый элемент меньше либо равен родителю, а корнем дерева является максимальный элемент массива.

На рис. 4.22 приведен пример пирамиды, имеющей глубину 4.

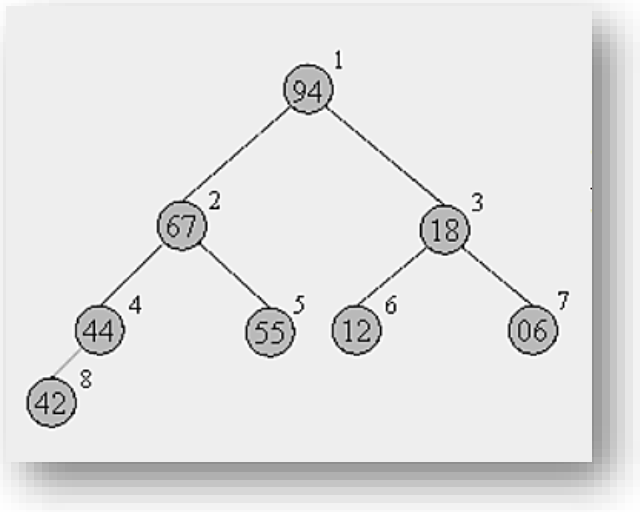


Рис. 4.22. Пример пирамиды

Для представления пирамиды не требуется создавать специальную структуру данных – ее можно хранить прямо в сортируемом массиве. Соответствие между геометрической структурой пирамиды как дерева и массивом устанавливается по следующей схеме: в *a[0]* хранится корень дерева, левый и правый сыновья элемента *a[i]* хранятся в *a[2i+1]* и *a[2i+2]* соответственно.

Для массива, хранящего в себе пирамиду, выполняется следующее характеристическое свойство: $a[i] \geq a[2i+1]$ и $a[i] \geq a[2i+2]$.

Плюсы такого хранения пирамиды очевидны:

- никаких дополнительных переменных, нужно лишь понимать схему;
- узлы хранятся от вершины и далее вниз, уровень за уровнем;
- узлы одного уровня хранятся в массиве слева направо.

Запишем в виде массива пирамиду, изображенную на рис. 4.22. Слева - направо, сверху - вниз: 94, 67, 18, 44, 55, 12, 06, 42. На рисунке место элемента пирамиды в массиве обозначено цифрой вверху и справа от него.

Восстановить пирамиду из массива как геометрический объект легко – достаточно вспомнить схему хранения и нарисовать, начиная от корня.

Пирамидальная сортировка включает две фазы: приведение исходного массива к виду пирамиды (построение пирамиды) и собственно сортировка. Рассмотрим эти фазы подробнее.

Фаза построения пирамиды. Начинать построение пирамиды можно с *a[l]...a[n – 1]*, где *l = n div 2*. Эта часть массива удовлетворяет свойству пирамиды, так как не существует индексов *i* и *j*, таких, что *i = 2l+1* и *j = 2l+2*, просто потому, что такие *i* и *j* находятся за границей массива.

Следует заметить, что неправильно говорить о том, что *a[l] ... a[n – 1]* является пирамидой как самостоятельный массив. Это, вообще говоря, не верно: его элементы могут быть любыми. Свойство пирамиды сохраняется лишь в рамках исходного, основного массива *a[0] ... a[n – 1]*.

Далее будем расширять часть массива, добавляя по одному элементу за шаг. Следующий элемент на каждом шаге добавления – тот, который стоит перед уже готовой частью.

Чтобы при добавлении элемента сохранялась пирамидальность, будем использовать следующую процедуру расширения пирамиды *a[k+1]...a[n – 1]* влево на новый элемент *a[k]:*

1) смотрим на потомков *a[k]* слева и справа (если он существует) – в массиве это *a[2k+1*] и а*[2k+2]* и выбираем наибольшего из них.

2) если этот элемент больше *a[k]* – меняем его с *a[k]* местами и идем к шагу 1, имея в виду новое положение *a[k]* в массиве. Иначе конец процедуры.

Таким образом, новый элемент *a[k]* как бы "просеивается" сквозь пирамиду.

На рис. 4.23 а) приведена блок-схема процедуры просеивания элемента сквозь пирамиду (*downHeap*). Входными параметрами этой процедуры являются: *{a[i]}* – преобразуемый массив, *k* – индекс просеиваемого элемента, *ub* – индекс последнего элемента массива.

Применяя процедуру просеивания к элементам *a[l-1]*, *a[l-2],…, a[0]* получим пирамиду. На рис. 4.23 б) приведена блок-схема алгоритма построения пирамиды (*MakeHeap*).

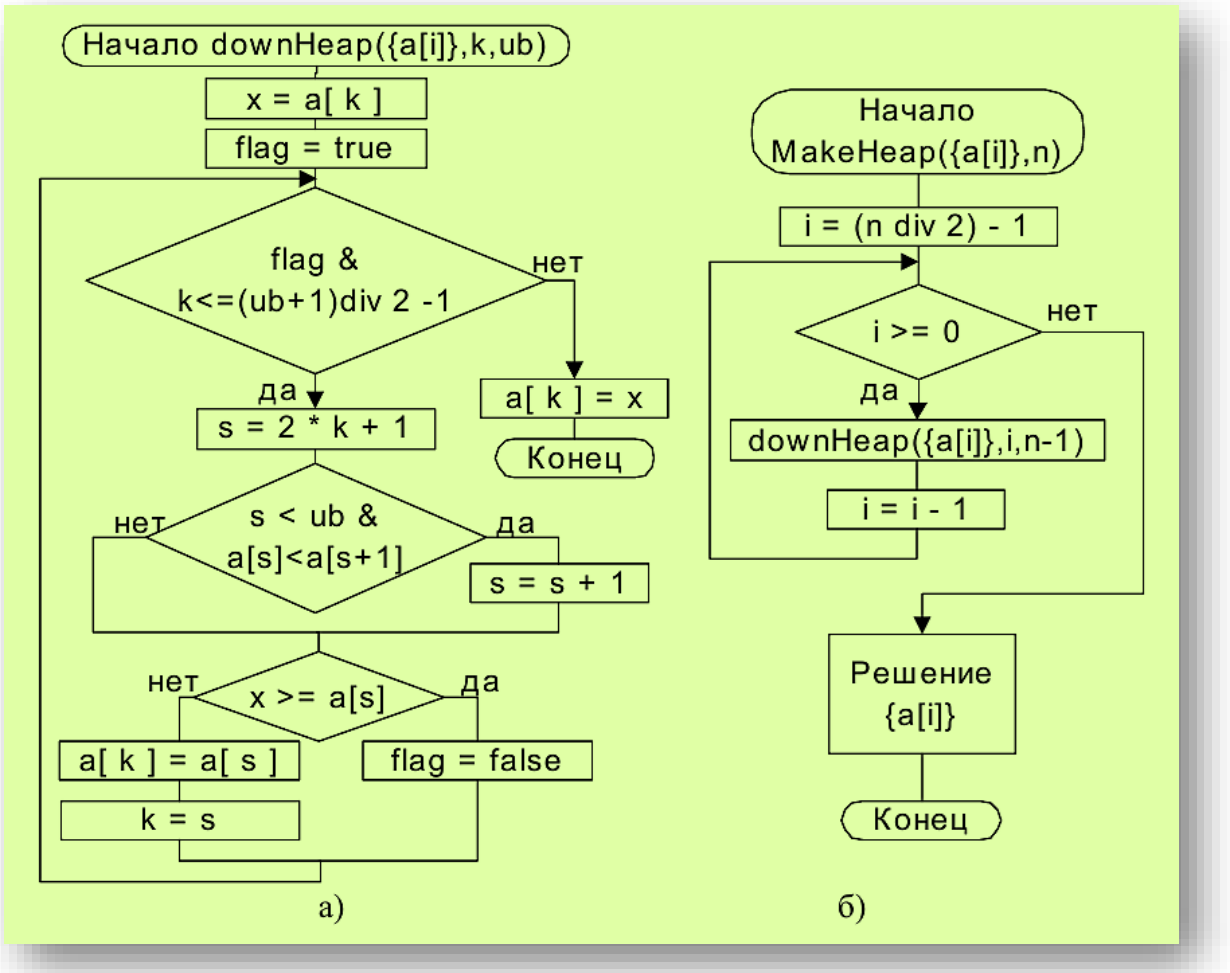


Рис. 4.23. Блок-схемы процедур приведения массива к пирамидальному виду:

а) процедура "просеивания" элемента сквозь пирамиду

б) алгоритм построения пирамиды

Рассмотрим пример построения пирамиды из исходного массива: 44, 55, 12, 42, 94, 18, 6, 67. В геометрической интерпретации элементы a[n div 2] ... a[n-1] (94, 18, 6, 67) является листьями в бинарном дереве. Один за другим остальные элементы продвигаются на свои места, и так, пока не будет построена вся пирамида. На рис. 4.24 изображен процесс построения пирамиды. Неготовая часть пирамиды (начало массива) окрашена в белый цвет, удовлетворяющий свойству пирамиды конец массива – в темный.

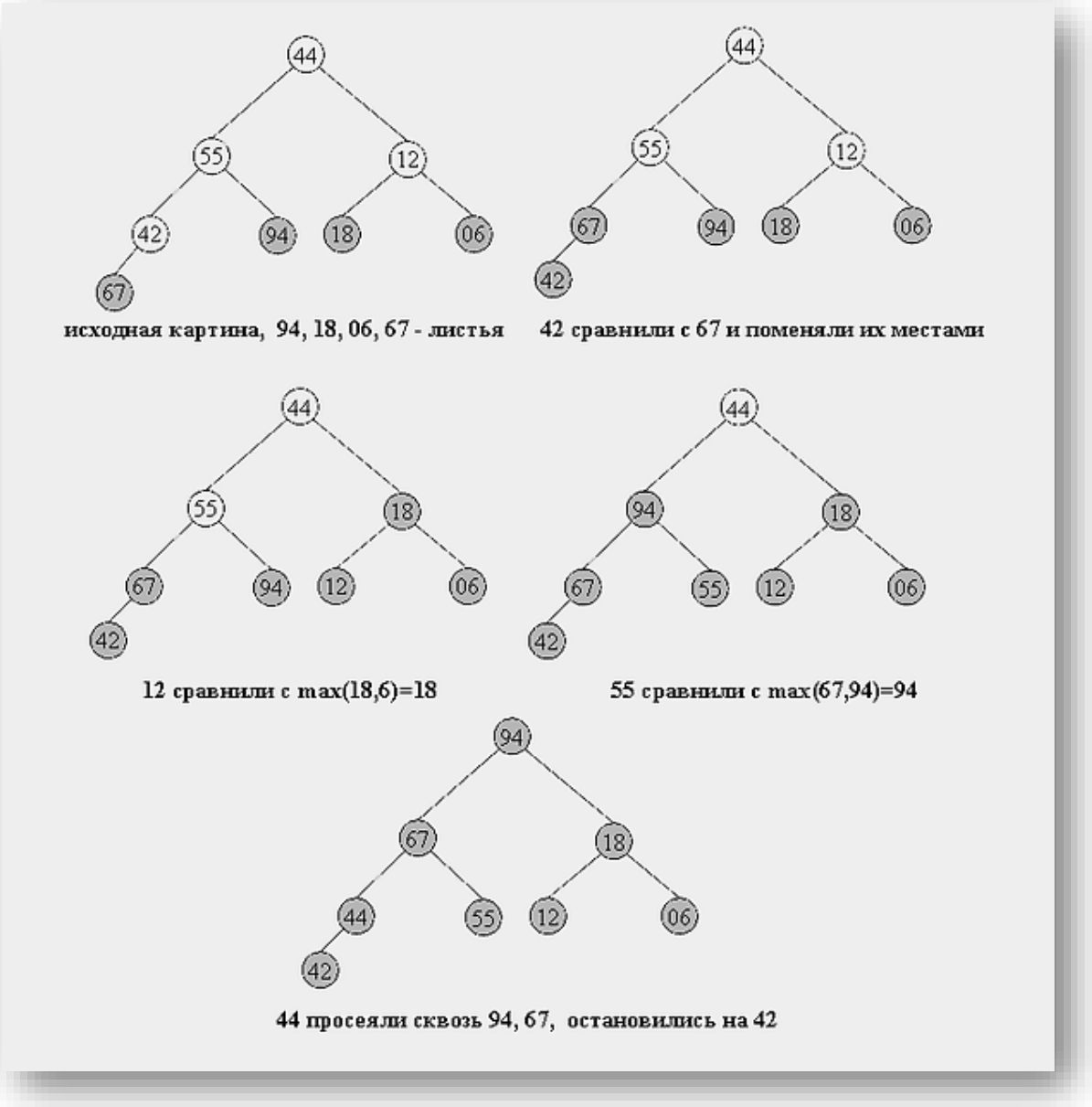


Рис. 4.24. Иллюстрация построения пирамиды

Фаза сортировки. Как видно из свойств пирамиды, в корне всегда находится максимальный элемент. Отсюда вытекает алгоритм сортировки:

1) берем верхний элемент пирамиды a[0]...a[n-1] (первый в массиве) и меняем с последним местами. Теперь "забываем" об этом элементе и далее рассматриваем массив a[0]...a[n – 2]. Для превращения его в пирамиду достаточно просеять лишь новый первый элемент;

2) повторяем шаг 1, пока обрабатываемая часть массива не уменьшится до одного элемента.

На рис. 4.25 приведен пример иллюстрирующий этот алгоритм.

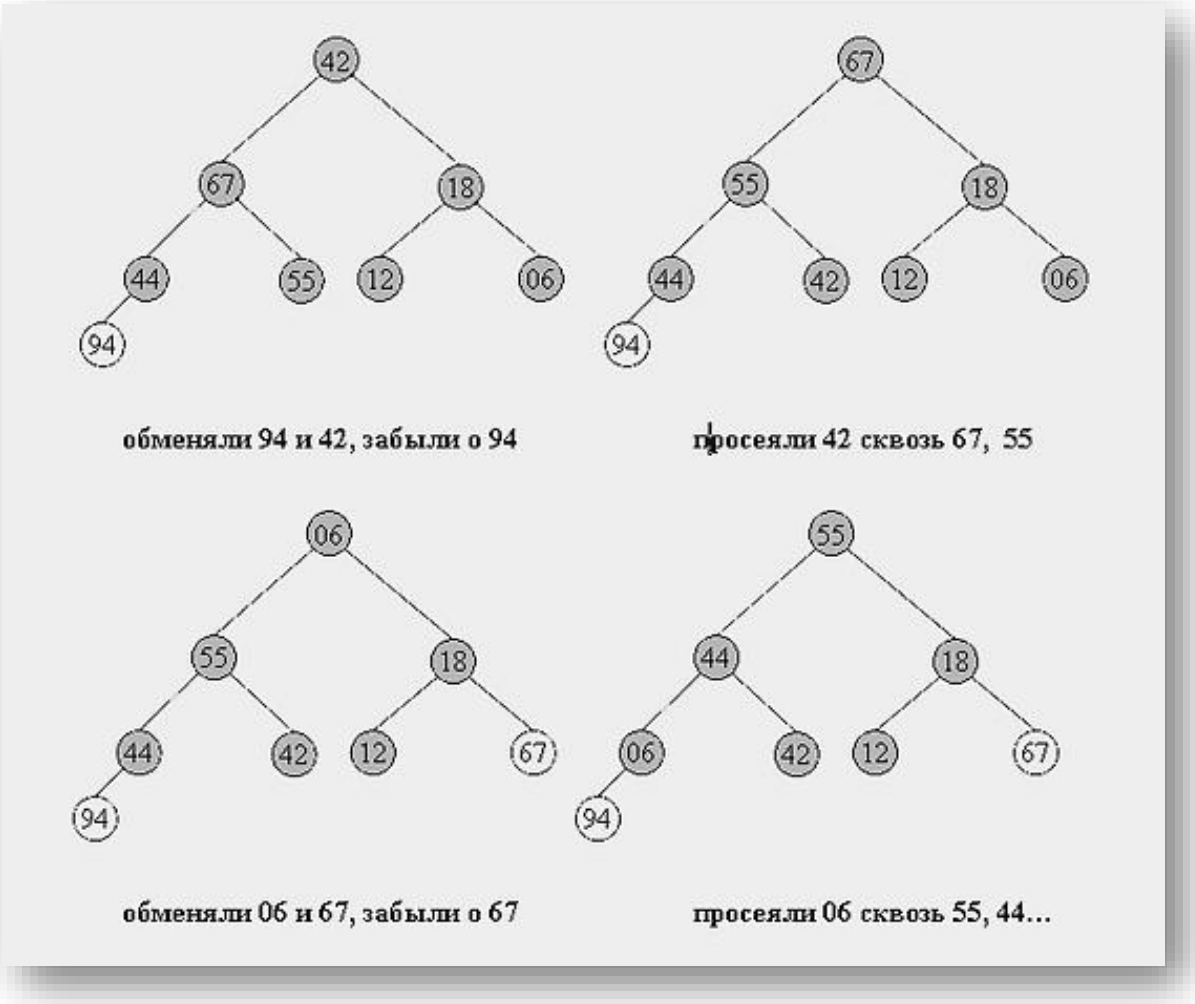


Рис. 4.25. Иллюстрация алгоритма пирамидальной сортировки

Очевидно, в конец массива каждый раз попадает максимальный элемент из текущей пирамиды, поэтому в правой части постепенно возникает упорядоченная последовательность.

На рис. 4.26 приведена блок-схема алгоритма пирамидальной сортировки (*HeapSort*).

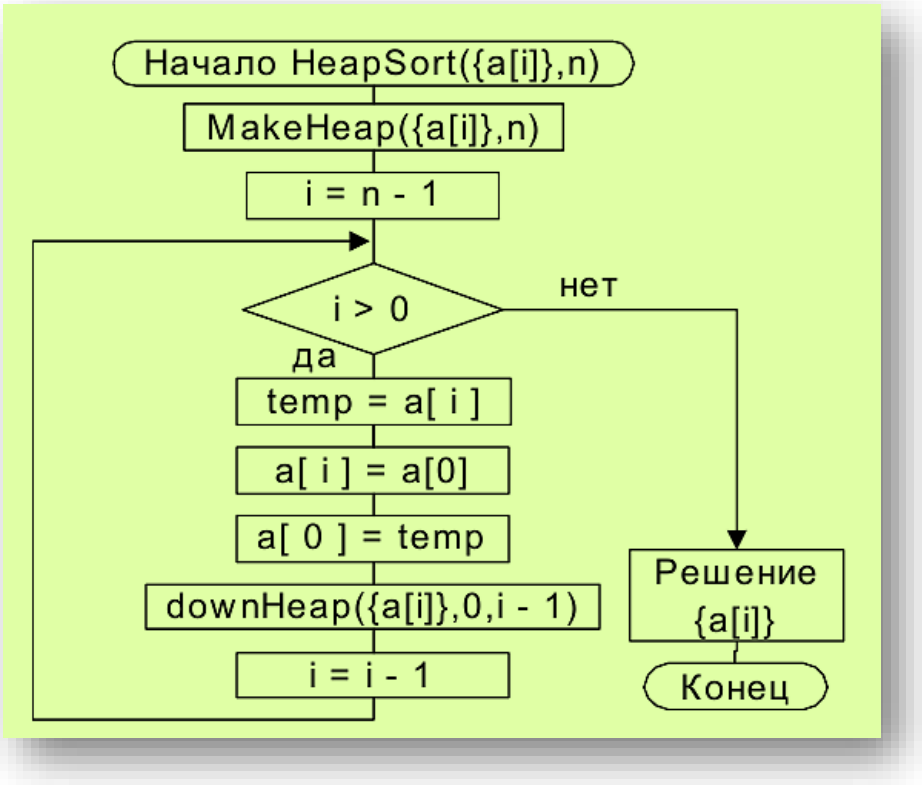


Рис. 4.26. Блок-схема алгоритма пирамидальной сортировки

Каково быстродействие получившегося алгоритма? Учитывая, что высота пирамиды не более $log_2 n$, процедура *downHeap* требует времени исполнения порядка $O(\log n)$. Построение пирамиды занимает порядка $O(n \log n)$ операций, причем более точная оценка дает даже $O(n)$ за счет того, что реальное время выполнения *downHeap* зависит от высоты уже созданной части пирамиды.

Вторая фаза занимает $O(n \log n)$ времени: $O(n)$ раз берется максимум и происходит просеивание бывшего последнего элемента. Плюсом является стабильность метода: среднее число пересылок $(n \log n)/2$, и отклонения от этого значения сравнительно малы.

Пирамидальная сортировка не использует дополнительной памяти. Метод не является устойчивым: по ходу работы массив так "перетряхивается", что исходный порядок элементов может измениться случайным образом. Поведение неестественно: частичная упорядоченность массива никак не учитывается.